



Pearls AQI Predictor

End-to-End Air Quality Forecasting and Monitoring System

Rabia Noreen

Data Science Intern

6 September 2026

Contents

1 Project Overview	ii
2 Problem Statement	ii
3 Objectives	ii
4 Technical Architecture	ii
4.1 Data Sources	iii
4.2 Main Workflow	iii
4.3 System Components	iii
5 Data and Feature Engineering	iii
6 Modeling Strategy	iv
6.1 Models Used	iv
6.2 Model Performance Results	iv
7 Prediction and Alerting	iv
8 Explainability	v
9 User Interface and API	v
9.1 Streamlit Dashboard	v
9.2 FastAPI Backend	v
10 Deployment and Runtime Design	v
10.1 Storage Strategy	vi
10.2 Runtime Behavior	vi
11 Why the Data Pipelines Must Run First	vi
12 Challenges and Limitations	vi
13 Conclusion	vi
14 Submission Summary	vii

1 Project Overview

Pearl AQI Predictor is an end-to-end machine learning and data engineering project that forecasts Air Quality Index (AQI) for major Pakistani cities. It combines weather data, historical AQI, feature engineering, model training, explainability, and visualization into one workflow.

The system answers a practical question: given current and recent environmental conditions, what AQI can be expected over the next 24 to 72 hours, and which factors influence the forecast? This can support public health awareness, environmental monitoring, and planning.

GitHub: <https://github.com/Rabia605/pearls-aqi-predictor>

Live Demo: <https://pearls-aqi-predictor-pakistan.streamlit.app>

2 Problem Statement

Air pollution is a major public health concern in urban areas, where traffic, industry, weather, and seasonal changes can increase pollutant levels. Since AQI can change quickly, residents and institutions may struggle to anticipate unhealthy conditions. The project addresses this by automating a pipeline that:

- gathers air quality and weather data,
- creates predictive features from recent observations,
- trains forecasting models,
- stores features and predictions,
- provides results through a dashboard and API,
- and identifies hazardous air-quality conditions.

3 Objectives

The primary objectives are to:

1. Build a reliable AQI forecasting system for selected Pakistani cities.
2. Forecast AQI at 24h, 48h, and 72h horizons.
3. Maintain continuously updated feature and prediction stores.
4. Provide explainable forecasts showing factors affecting AQI predictions.
5. Deliver results through an intuitive dashboard and API.
6. Support cloud storage with local fallback storage for reliable operation.

4 Technical Architecture

The project uses a decoupled architecture in which each pipeline has a defined role. Data is stored in a feature store with local fallback support.

4.1 Data Sources

The project uses multiple external and internal sources:

- AQICN for current air-quality metrics,
- Open-Meteo for historical and forecast weather data,
- OpenWeather as an additional weather source,
- Local city metadata and configuration from the repository.

4.2 Main Workflow

1. The feature pipeline retrieves recent AQI and weather data.
2. Feature engineering creates lags, rolling averages, weather patterns, and time-based signals.
3. Processed data is stored in the feature store or parquet fallback.
4. The training pipeline reads the features and trains forecasting models.
5. The inference pipeline generates 24h, 48h, and 72h forecasts.
6. Forecasts are saved and used by the API and Streamlit dashboard.
7. Alert logic identifies unhealthy or hazardous AQI conditions.
8. Explainability tools identify the main prediction drivers.

4.3 System Components

- Machine learning pipelines under the `ml` package
- FastAPI backend under `api`
- Streamlit dashboard under `streamlit_app`
- Local data persistence in the `data` folder
- Configuration through `.env` and project settings

5 Data and Feature Engineering

The predictive system uses feature engineering to transform historical AQI and weather data into useful forecasting variables.

Examples include:

- lagged AQI values,
- rolling means and moving averages,
- cyclic time features such as hour, month, and weekday,
- temperature, humidity, and wind metrics,
- pollutant indicators such as PM2.5,

- environmental patterns showing worsening or improving air conditions.

These features convert time-series behavior into a supervised learning problem suitable for tabular machine learning.

6 Modeling Strategy

The project uses multiple models to forecast AQI and provide performance benchmarks.

6.1 Models Used

- Ridge Regression
- Random Forest Regressor
- LSTM-based deep learning model

6.2 Model Performance Results

Model	Horizon	RMSE	MAE	R ²
Ridge Regression	24h	19.406	12.667	0.770
Random Forest	24h	20.146	12.669	0.752
Persistence Baseline	24h	24.347	14.295	0.637
Ridge Regression	48h	25.155	17.259	0.615
Random Forest	48h	27.512	19.232	0.539
Persistence Baseline	48h	29.692	18.756	0.463
Ridge Regression	72h	26.416	18.798	0.575
Random Forest	72h	28.610	20.423	0.502
Persistence Baseline	72h	31.377	20.939	0.401

Ridge Regression achieved an R² of 0.770 at 24h, explaining 77% of AQI variance and outperforming Random Forest and the baseline. At 72h, Ridge achieved an R² of 0.575 compared with 0.401 for the baseline, representing a 43% relative improvement in explained variance.

7 Prediction and Alerting

The system forecasts AQI across multiple horizons to show short-, medium-, and longer-term pollution conditions. Generated outputs include:

- current AQI context,
- AQI forecasts for the next 24, 48, and 72 hours,
- model benchmark comparisons,
- hazardous-condition indicators,
- and alerts when forecasts exceed dangerous thresholds.

8 Explainability

The project not only predicts AQI but also explains the predictions. SHAP (SHapley Additive exPlanations) is used to show which factors contribute to increases or decreases in forecasted AQI. This improves transparency and helps users understand the model's decision logic, making the system more useful for practical decision support.

9 User Interface and API

The system provides both an interactive dashboard and a REST API for accessing AQI forecasts and related information.

9.1 Streamlit Dashboard

The dashboard provides an interactive interface for exploring AQI conditions and viewing forecasts.

- city-based AQI exploration,
- compact forecast cards,
- AQI trend charts,
- diagnostic and model comparison views,
- health warnings and recommendations,
- and a polished air-quality monitoring interface.

9.2 FastAPI Backend

The backend provides endpoints for:

- service health checks,
- city information,
- latest AQI values,
- multi-horizon forecasts,
- model benchmark data,
- and forecast metadata.

10 Deployment and Runtime Design

The system is designed for reliable deployment, data storage, and runtime operation across cloud and local environments.

10.1 Storage Strategy

The project uses Hopsworks for feature storage and model registry access when available, with a local parquet fallback. This allows the system to continue operating when cloud credentials or external services are unavailable.

10.2 Runtime Behavior

The app reads feature data before rendering. If no feature rows exist, it displays a message asking the user to run the data pipelines first. This is an intentional data-dependency check, not an application bug.

11 Why the Data Pipelines Must Run First

This is a key operational point. The project follows a data-first architecture: no features means no meaningful forecast page, no forecast data means no actionable predictions, and no training run means no benchmark registry.

- No features $\Rightarrow$ no meaningful forecast page
- No forecast data $\Rightarrow$ no actionable predictions
- No training run $\Rightarrow$ no benchmark registry
- No model data $\Rightarrow$ no model-serving context

The feature pipeline must be run before using the dashboard in a fresh environment. Therefore, the message *"Run the data pipelines first"* appears when no feature data is available.

12 Challenges and Limitations

Several real-world challenges remain:

- External APIs may fail or impose rate limits.
- Hopsworks cloud access may be unavailable locally.
- Feature storage requires a completed data pipeline.
- AQI forecasting is sensitive to seasonal and environmental changes.
- Model accuracy depends on data quality and feature availability.

These challenges are partly addressed through the fallback store and modular pipeline design.

13 Conclusion

Pearl AQI Predictor is an applied AI project combining data engineering, machine learning, explainability, and product delivery into one system. It forecasts air quality while making predictions accessible through an API and dashboard. The project demonstrates modular

pipelines, versioned data handling, model benchmarking, storage fallback, and user-facing deployment. It shows how environmental data can be transformed into a practical decision-support system for technical and non-technical users. The key operational lesson is that the system depends on generated data, so the pipelines must run before meaningful forecasts can be displayed. Once completed, the project provides a usable AQI prediction platform.

14 Submission Summary

This submission presents a complete AQI forecasting solution with:

- data ingestion,
- feature engineering,
- model training,
- inference generation,
- alert detection,
- explainability,
- API access,
- and dashboard visualization.

GitHub: <https://github.com/Rabia605/pearls-aqi-predictor>

Live Demo: <https://pearls-aqi-predictor-pakistan.streamlit.app>